Лабораторна робота №5
Технології DevOps
GitOps: розгортання проєкту через Argo CD
Інструкція для студента

Скрізь нижче `myapp` — назва вашого проєкту і чарту, `myapp-gitops` — репозиторій конфігурації, `<user>` — ваш логін на GitHub або GitLab.

---

1. Репозиторій конфігурації

Рекомендована структура (однакова для обох варіантів, різниця лише в тому, в якому репозиторії вона лежить):

```
myapp-gitops/
├── bootstrap/
│   └── root.yaml               # кореневий Application, застосовується вручну один раз
├── argocd-apps/                # Application-и, якими керує root
│   └── myapp.yaml
└── charts/
    └── myapp/                  # чарт з ЛР-4 (Chart.yaml, values.yaml, templates/ ...)
        └── values-minikube.yaml
```

Якщо обираєте варіант із тим самим репозиторієм застосунку, обмежте CI-пайплайн фільтром шляхів, щоб зміни в `bootstrap/`, `argocd-apps/` і `charts/` не запускали збірку образу (`on.push.paths-ignore` у GitHub Actions, `rules: changes` у GitLab CI).

Секрети. Найпростіший спосіб — не тримати Secret у чарті для цього середовища. Додайте в чарт параметр `existingSecret`: якщо він заданий, шаблон Secret не рендериться, а Deployment посилається на Secret, створений вручну:

```
# templates/secret.yaml
{{- if not .Values.existingSecret }}
... ваш шаблон Secret ...
{{- end }}

# у Deployment (envFrom / secretKeyRef):
name: {{ .Values.existingSecret | default (printf "%s-secret" (include "myapp.fullname" .)) }}
```

```
# values-minikube.yaml
existingSecret: myapp-secret

kubectl create namespace myapp
kubectl -n myapp create secret generic myapp-secret \
  --from-literal=DB_PASSWORD=... --from-literal=API_TOKEN=...
```

Argo CD не видалятиме такий Secret навіть з `prune`, бо він не створений із Git. Повноцінні рішення (Sealed Secrets, SOPS, External Secrets) — за бажанням.

2. Образ застосунку в реєстрі

Додайте до застосунку відображення версії, найпростіше через змінну оточення, яку задає чарт (`APP_VERSION`), або константу в коді. Приклад для Python/Flask:

```
@app.get("/version")
def version():
    return {"version": os.getenv("APP_VERSION", "dev")}
```

Зберіть і опублікуйте образ (або використайте пайплайн з попередніх лабораторних):

```
docker build -t ghcr.io/<user>/myapp:1.0.0 .
docker push ghcr.io/<user>/myapp:1.0.0
```

Пакет у GHCR за замовчуванням приватний: зробіть його публічним у налаштуваннях пакета, інакше кластер отримає `ImagePullBackOff`. У `values-minikube.yaml` вкажіть

`image.repository`, `image.tag: "1.0.0"` і `pullPolicy: IfNotPresent`.

3. Встановлення Argo CD

```
minikube start
kubectl create namespace argocd
kubectl apply -n argocd --server-side --force-conflicts \
  -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl -n argocd get pods -w            # дочекатися Running

# CLI: macOS
brew install argocd
# Linux
curl -sSL -o argocd \
  https://github.com/argoproj/argo-cd/releases/latest/download/argocd-linux-amd64
sudo install -m 555 argocd /usr/local/bin/argocd && rm argocd
# Windows: argocd-windows-amd64.exe зі сторінки релізів, перейменувати в argocd.exe, додати в PATH
```

```
kubectl -n argocd port-forward svc/argocd-server 8080:443    # окремий термінал
argocd admin initial-password -n argocd
argocd login localhost:8080 --username admin --insecure
argocd account update-password
kubectl -n argocd delete secret argocd-initial-admin-secret
```

Інтерфейс: https://localhost:8080 (сертифікат самопідписаний, браузер попередить). Публічний репозиторій нічого додатково не потребує. Для приватного:

```
argocd repo add https://github.com/<user>/myapp-gitops.git \
  --username <user> --password <token з правом читання репозиторію>
```

4. Застосунок під керуванням Argo CD

`argocd-apps/myapp.yaml`. Назва `Application` стає назвою Helm-релізу, тож імена об'єктів будуть як у ЛР-4 при `helm install myapp`:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: myapp
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/<user>/myapp-gitops.git
    targetRevision: main
    path: charts/myapp
    helm:
      valueFiles:
        - values-minikube.yaml        # шлях відносно каталогу чарту
  destination:
    server: https://kubernetes.default.svc
    namespace: myapp
  syncPolicy:
    automated:
      prune: true                     # selfHeal додамо в п. 6
    syncOptions:
      - CreateNamespace=true
```

`bootstrap/root.yaml` — кореневий застосунок, який стежить за каталогом `argocd-apps/`:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: root
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/<user>/myapp-gitops.git
    targetRevision: main
    path: argocd-apps
  destination:
    server: https://kubernetes.default.svc
```

```
    namespace: argocd
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

```
git add . && git commit -m "Deploy myapp via Argo CD" && git push

kubectl apply -f bootstrap/root.yaml         # єдиний ручний apply
argocd app list
argocd app get myapp
kubectl -n myapp get pods
helm list -n myapp                           # порожньо: Helm лише рендерить шаблони
```

Надалі новий застосунок чи компонент кластера додається одним файлом в `argocd-apps/` — так у ЛР-6 і ЛР-7 з'являться Argo Rollouts і стек моніторингу.

5. Оновлення через Git

Внесіть видиму зміну (версія, текст відповіді), зберіть образ `1.1.0`, опублікуйте. Далі лише Git:

```
# у charts/myapp/values-minikube.yaml: tag: "1.0.0" → tag: "1.1.0"
git commit -am "Release myapp 1.1.0" && git push

argocd app get myapp --refresh       # не чекати опитування Git (до 3 хв)
kubectl -n myapp get pods -w

kubectl -n myapp port-forward svc/myapp 8081:80
curl localhost:8081/version
```

В інтерфейсі те саме: кнопка Refresh, потім видно `OutOfSync`, автоматичну синхронізацію і нові поди в дереві. У реальному проєкті коміт зі зміною тегу робить CI після збірки образу.

6. Дрейф, Health і відкат

a) Дрейф. Поки `selfHeal` вимкнено:

```
kubectl -n myapp scale deployment myapp --replicas=5
argocd app get myapp            # OutOfSync, але репліки лишаються 5
```

Додайте `selfHeal: true` до `syncPolicy.automated` у `argocd-apps/myapp.yaml`, запуште (root застосує зміну сам) і повторіть `kubectl scale` — Argo CD поверне кількість реплік за кілька секунд.

b) Synced, але Degraded. Змініть тег на неіснуючий (`9.9.9`), запуште:

```
argocd app get myapp --refresh
# Sync Status: Synced  — маніфест у кластері такий самий, як у Git
# Health Status: Degraded / Progressing — нові поди в ImagePullBackOff
kubectl -n myapp get pods
```

c) Відкат. Повертаємо опис, а не кластер:

```
git log --oneline
git revert <коміт з 9.9.9> && git push
argocd app get myapp --refresh
argocd app history myapp        # кожна синхронізація прив'язана до коміту
curl localhost:8081/version
```

`argocd app rollback` працює лише з вимкненою автосинхронізацією: інакше Argo CD одразу поверне стан із Git. Тому відкочуємо комітом.

7. Захист: лайв-демонстрація

Перед захистом запустіть minikube, port-forward до Argo CD і застосунку, підготуйте образи двох версій. Рекомендований порядок показу:

- інтерфейс Argo CD: `root` → `myapp` → дерево об'єктів, `Synced` / `Healthy`;
- зміна тегу → commit → push → Refresh → нова версія в `curl /version`;

- `kubectl scale` → Argo CD повертає репліки (`selfHeal`);
- неіснуючий тег → `Synced` + `Degraded` → `git revert` → `Healthy`.

Будьте готові до прохання викладача змінити інше значення (кількість реплік, змінну оточення) і показати, як воно доїде до кластера.

Типові помилки

- `ComparisonError: ... repository not found`**.** Репозиторій приватний і не доданий через `argocd repo add`, або помилка в `repoURL`.
- `ImagePullBackOff`**.** Образ приватний або є лише локально в Docker. Зробіть пакет публічним або додайте `imagePullSecrets`.
- `values-minikube.yaml: no such file`**.** `valueFiles` шукаються відносно `path` чарту, а не кореня репозиторію.
- `Application` **у неправильному namespace.** Дочірні `Application` мають бути в `argocd`, інакше Argo CD їх не бачить.
- **Зміна не приїхала.** Запушено не в ту гілку, що в `targetRevision`, або ще не минув інтервал опитування — натисніть Refresh.
- **Ручне** `kubectl apply` **чарту поруч з Argo CD.** Два джерела правди: зміни будуть перетерті або розійдуться з Git.